CONVERGE · THE CANONICAL BLUEPRINT · AUTONOMOUS SYSTEMS

Converge

The Seven Passes

The canonical blueprint for compiling a fuzzy brief into an autonomous, eval-gated system — one fixed descent, seven passes, each ending at a gate.

Document type	Canonical process blueprint — the repeatable steps, not the deck
Source	cvg-aut-systems-spine (the 7-pass spine)
Worked example	a generic data-engineering build (medallion → serving)
The invariant	Every pass lowers altitude, binds an engine, ends at a gate
Converged means	An eval passed — not that you feel done

This document states the canonical sequence and the gate that closes each pass. The passes do not change — only who stands at the gate. Today it is you; the dark factory replaces each human gate with a runnable eval.

Introduction

Why Converge exists

The old loop was: a human reads a fuzzy requirement, holds the whole thing in their head, and writes the system — deciding, at every step, when a piece is "done." That loop does not scale to AI engines, because "done" was never written down. It lived in the engineer's judgment. An AI-native engineer does not write the system; they **lower a fuzzy requirement through a repeatable descent** until a machine can execute it without supervision.

The main idea — compile intent, don't write the system

Converge treats building like compilation. You start at the top with raw intent and lower it, one level of altitude at a time, until what remains is concrete enough for an engine to run: **intent** → **structure** → **plan** → **tasks** → **code**. Each step is closer to something a machine executes. You are not the author of the final system — you are the compiler that turns a brief into it.

EVERY PASS

Lowers altitude

intent → structure → plan → tasks → code — each step closer to something a machine runs.

BINDS

An engine

the right tool for the altitude — Chat to think, Code to build, Codex to attack, Kimi to crank.

ENDS AT

A gate

converged means an eval passed — not that you feel done. The gate is the unit of trust.

What "canonical" means here

Canonical means the sequence is fixed and the gates are non-negotiable: the same seven passes, in the same order, each closing on the same kind of test — regardless of project. **Blueprint** means this document is the thing you build *from*: each pass below names its altitude, its engine, its input and output, a concrete example on this repo's stack, and the gate that must pass before the next pass begins.

Today the gate is you; tomorrow it is an eval

Right now a human stands at each gate — you confirm, you approve, you review. The endpoint of Converge is the **dark factory**: each human gate is replaced by a runnable eval, and the human steps out one gate at a time. The passes do not change; only who stands at the gate.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. **You are converged when the eval passes — not when you feel done.**

Contents

0 · The Whole Method, On One Line

Intent → Structure → Decomposition → Consensus → **[Specify | Tasking]** → Harness → Execution. Each output is the next input. After Consensus the path forks — **5A Specify** (plan-driven) or **5B Tasking** (task-driven) — then reconverges at Harness. The chain descends from a fuzzy brief to merged code. **You are converged when the eval passes — not when you feel done.**

#	PASS	ENGINE	OUT / GATE
1	**Intent**	`Claude CoWork`	tech-spec PDF · spec answers brief
2	**Structure**	`Claude Code · repo`	no file · system understood
3	**Decompose**	`Claude Code · auto`	swimlane plans · one per seam
4	**Consensus**	`Codex adversary`	plans sharpened · fork decided
■	*The Fork*	`decision`	where does the trust boundary sit?
5A	*Specify (plan-driven)*	`SpecKit / OpenSpec`	one coherent spec · end-to-end eval
5B	*Tasking (task-driven)*	`task-spec skill`	many atomic units · per-unit evals
6	**Harness**	`agents-kbs skill`	.claude/ · control layer up
7	**Execute**	`Workflows · Kimi`	merged code · eval = merged

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. Three guardrails make the sequence **canonical**, not merely descriptive: **Pass 2 produces no file**, **Pass 3 is plan altitude only**, and **the fork is an output of Pass 4** — not a free-floating decision.

The spine, drawn two ways

The same method, drawn two ways. First, the seven passes as a flow — each ending at its gate. Then the altitude the work falls through — intent narrows to code, one rung at a time.

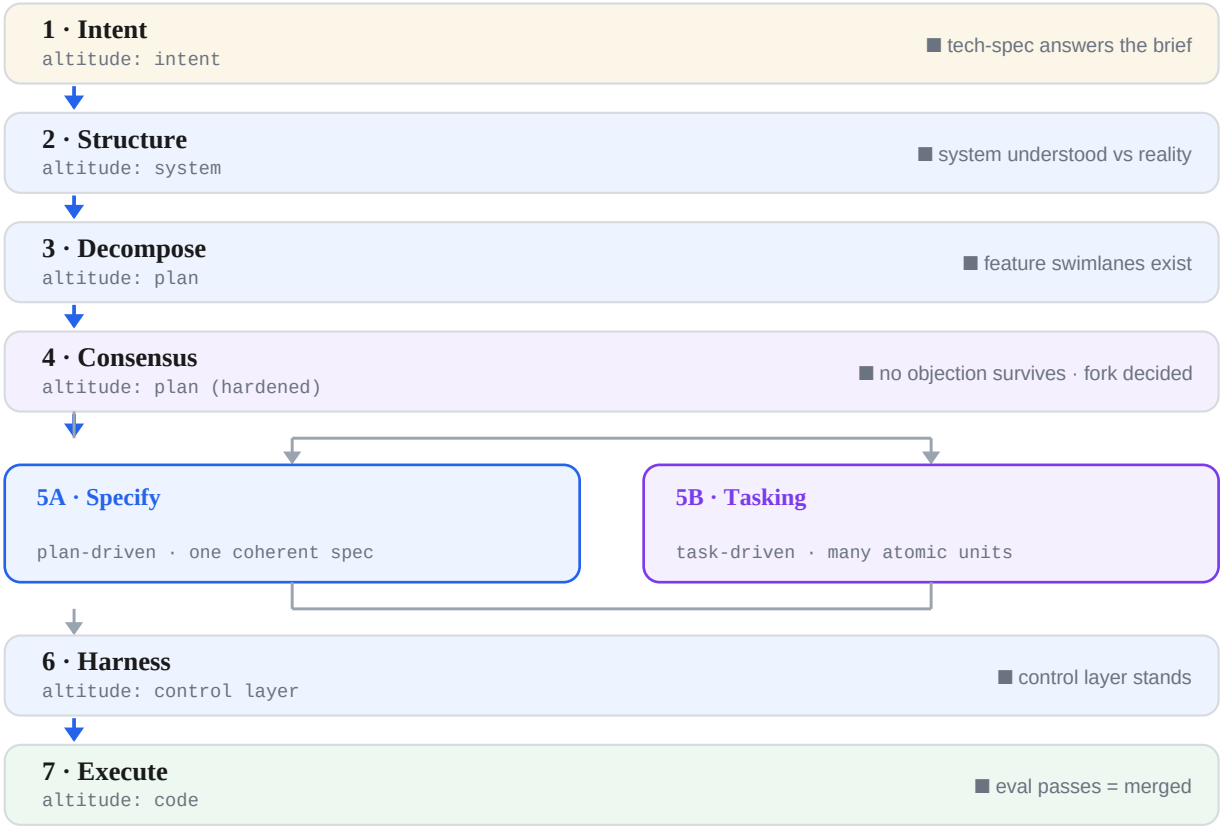

FIG 1 The descent — each pass lowers altitude, binds an engine, ends at a gate. After Consensus the path forks (5A Specify / 5B Tasking) and reconverges at Harness.

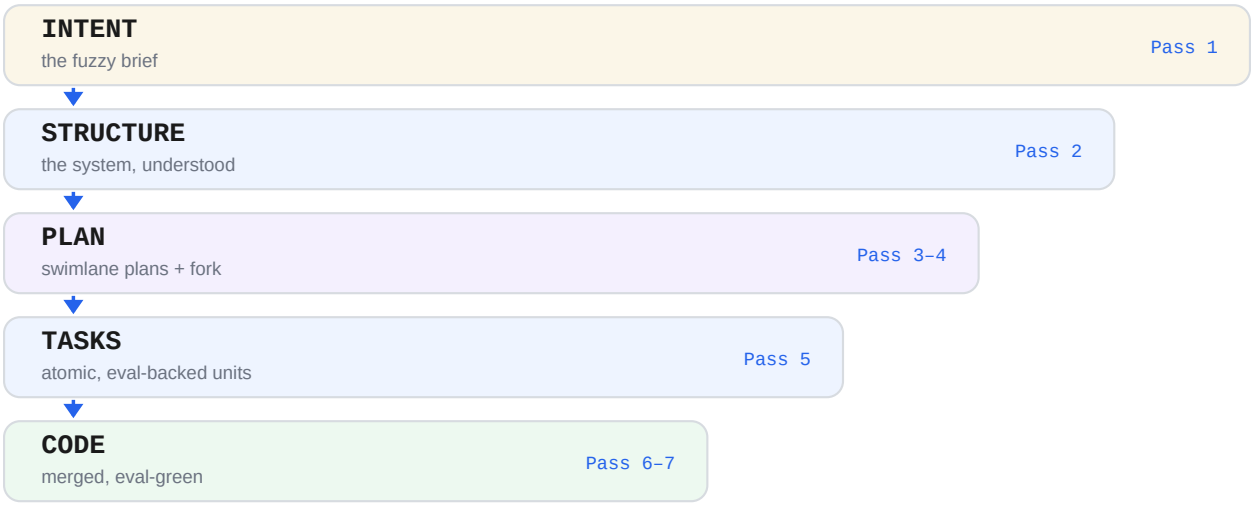

FIG 2 The altitude ladder — intent → structure → plan → tasks → code; the bar narrows as the work falls toward something a machine runs.

The same brief, descending through all seven passes

To make the phases concrete, here is one generic data-engineering brief — *"stand up an analytical layer the business can query, separate from the operational system"* — lowered through the canonical descent. Each row is one altitude drop; the example in each row is what that pass actually produces, in vendor-neutral terms.

PASS	ALTITUDE	WORKED EXAMPLE — GENERIC DATA-ENGINEERING
1 · Intent	Intent	Brief in a CoWork project → tech-spec: "an analytical layer (bronze→silver→gold) over the operational data, exposed through a serving API."
2 · Structure	System	Open the existing codebase (brownfield) — or confirm there is none (greenfield); check the spec against real sources and schemas. No file; loaded understanding.
3 · Decompose	Plan	Swimlane plans along the natural seams — e.g. a transformation lane (bronze→silver→gold) and a serving lane (the API/BI layer over gold).
4 · Consensus	Plan (hardened)	A different model attacks every plan vs the tech-spec; every objection FIXED or ACCEPTED with an owner. The fork is decided here.
The Fork	Decision	Where does the trust boundary sit — one coupled system (plan-driven), or many independent units (task-driven)? The next step takes one of two forms.
5A · Specify	Coupled spec	PLAN-DRIVEN — plans → SpecKit / OpenSpec / AgentSpec → one coherent spec; verify with one end-to-end eval (sources → gold → serving).
5B · Tasking	Atomic unit	TASK-DRIVEN — plans → atomic tasks (build the silver fact model; build one serving endpoint), each with its own eval; verify per unit.
6 · Harness	Control layer	Both paths reconverge — the work names the tech (warehouse + transform + serving) → scaffold matching agents + KBs + rules for exactly those.
7 · Execute	Code	Crank engine builds each unit, a reviewer judges, an adversary checks; loop until the eval (end-to-end or per-unit) is green → merge.

PASS 1 OF 7 · ALTITUDE: INTENT

INTENT · RECEIVE & COMPREHEND THE BRIEF

The client hands you the problem. You produce the spec.

ENGINE · CLAUDE COWORK (PROJECT) | IN · THE CUSTOMER BRD | OUT · TECH-SPEC.PDF

Put **all** of the customer's material — the BRD, attachments, prior threads, constraints — into a single Claude CoWork project so the whole context lives in one place with persistent memory. Read it like a senior engineer, grill the few questions that most change how you build, then crystallize the technical answer. The pass ends by generating **tech-spec.pdf** — a consensus object you circulate before any code.

UNDERSTAND

Read

the BRD like a senior engineer — the real problem, who hurts, their numbers.

INTERROGATE

Grill

the 2–3 questions that most change how the system gets built.

CRYSTALLIZE

Spec

the technical answer to their problem — your consulting deliverable, born as a PDF.

GATE

You can state the client's problem back correctly, and the tech-spec **answers it**. The gate is the restatement + the answer — the PDF is the artifact, not the gate. No premature implementation the brief didn't justify.

PASS 2 OF 7 · ALTITUDE: SYSTEM

STRUCTURE · COMPREHEND THE SPEC AGAINST THE CODEBASE

Greenfield or brownfield — ground the spec in what exists.

ENGINE · CLAUDE CODE · ON THE REPO

IN · TECH-SPEC + EXISTING CODE

OUT · SESSION UNDERSTANDING (NO FILE)

First, name the ground you're on. **Brownfield** — an existing system you didn't write: open it, explain it end to end, and check the spec against what is actually there (schemas, sources, jobs). **Greenfield** — nothing exists yet: confirm that, and ground the spec in the real source systems and constraints it must honor instead. Either way, turn the tech-spec into real comprehension by asking questions, and produce a clean component + dependency map of what to build and in what order.

NAME THE GROUND

Field

brownfield (a system you didn't write) or greenfield (nothing yet) — the rest changes accordingly.

GROUND

Check

does the spec match reality — existing schemas and sources, or the constraints a new build must honor?

CONFIRM

Map

a clean component + dependency map — what to build, in what order.

GATE

You can explain the full system, consistent with the spec and what actually exists. **Pass 2 produces no file** — its output is the loaded understanding in the session. Keep it open into Pass 3: the understanding is the handoff. The moment you write an artifact here, you've drifted up into planning.

PASS 3 OF 7 · ALTITUDE: PLAN

DECOMPOSITION · BREAK IT INTO SWIMLANE PLANS

Split the work into focus-oriented swimlanes.

ENGINE · CLAUDE CODE (AUTO) · SAME SESSION

OUT · ONE SKETCH PLAN PER SWIMLANE

OUT · SWIMLANES BY FEATURE / COMPONENT

Carry the understanding from Pass 2 — without leaving the session — and split the work along its **natural seams**: by feature or by component. Each seam becomes a **swimlane**, and each swimlane becomes its own focus-oriented sketch plan. How many depends on the work — a transformation lane and a serving lane is the common two-lane case, but a system with several independent features splits into several lanes. Each plan lists features, dependencies, and a sane build order.

DECOMPOSE

Seams

find the natural seams — by feature or component — and justify each boundary.

SWIMLANE

One plan each

each swimlane becomes its own focus-oriented sketch plan; one lane, one plan.

COMMON CASE

Transform / serve

e.g. a medallion transformation lane and a serving lane — two of many possible swimlanes.

ON SWIMLANES

Swimlanes are how you stay **focus-oriented**: split the work by the seams that are real in *this* system — by feature or by component — and give each lane its own sketch plan. The right number is the number of genuine seams: often two (transform vs serve), sometimes more. The discipline is that each lane is independently planned and named; the canonical rule is **plan altitude only**, not a fixed lane count.

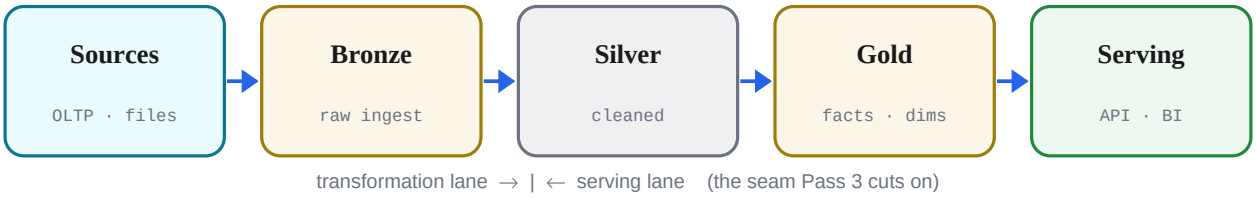

FIG 3 Reference architecture (the common transform → serve case) — one example of the seams Pass 3 splits into swimlanes; other systems split by feature or component.

GATE

One sketch plan per swimlane, each split by feature or component; each lists features, dependencies, and a sane build order. **Plan altitude only** — no atomic tasks, no code yet. The session carries from Pass 2; the understanding becomes the plans without leaving the session.

PASS 4 OF 7 · ALTITUDE: PLAN (HARDENED)

CONSENSUS · A DIFFERENT MODEL ATTACKS

Bring a different engine to refute the plan.

ENGINE · CODEX (ADVERSARY) → CLAUDE (REVISES)

IN · BOTH PLANS + THE TECH-SPEC

OUT · PLANS SHARPENED IN PLACE

Bring in a **different model** as an adversary. Codex attacks each plan as a skeptic who didn't write it — default to refuted. Check both plans against the tech-spec for contradictions and gaps, then revise: every objection is either **FIXED** in a plan or **ACCEPTED** with a named owner. Update the plans with the discoveries and improvements the attack surfaces.

ATTACK

Refute

Codex, as a skeptic who didn't write it — default to refuted.

GROUND

Drift?

check each plan against the tech-spec — contradictions, gaps.

SHARPEN

Fix / Own

every objection FIXED in a plan or ACCEPTED with an owner.

GATE

No open objection survives — nothing silently dropped. Cross-model disagreement is the point: Claude won't refute itself hard enough. **At the end of Consensus, the fork is decided** — does the trust boundary wrap the whole system, or each unit?

THE FORK · DECIDED AT THE END OF CONSENSUS

One coupled system, or many independent units?

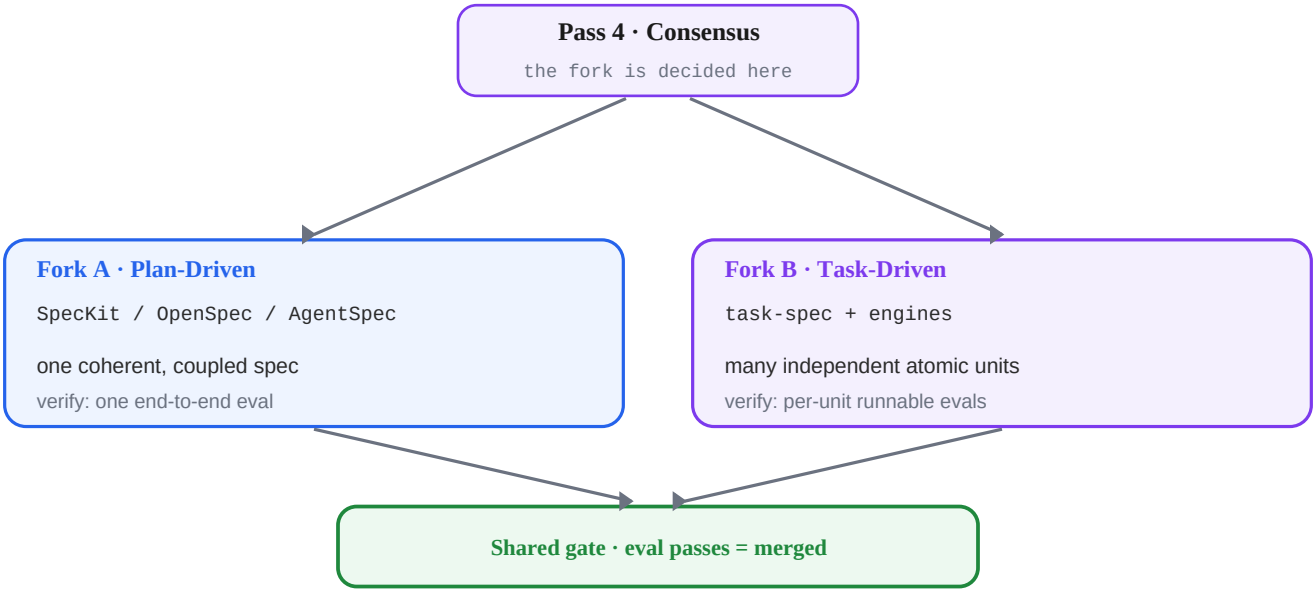

FIG 4 The fork branches at Consensus into two paths — and both reconverge on the one shared gate.

FORK A · PLAN-DRIVEN

One coherent spec

- SpecKit / OpenSpec / AgentSpec
- One tightly-coupled system — pieces only make sense together
- Trust boundary wraps the whole system
- Verify: one end-to-end test

FORK B · TASK-DRIVEN

Many atomic tasks

- task-spec + engines
- Loosely-coupled units — each stands alone (transform models, serving endpoints)
- Trust boundary wraps each unit
- Verify: per-unit runnable evals

BOTH FORKS RECONVERGE ON ONE GATE

eval passes = merged. The cut differs; the gate is identical. That shared gate is what makes Converge one method with two paths — and what lets either go dark-factory. **Both forks are canonical** — the next two sections develop each in full.

PASS 5A · SPECIFY · PLAN-DRIVEN · ONE COHERENT SPEC

ALTITUDE: COUPLED SPEC · TRUST BOUNDARY WRAPS THE WHOLE SYSTEM

5A · Specify — lift the swimlane plans into one coherent spec.

ENGINE · SPECKIT / OPENSPEC / AGENTSPEC

IN · THE TWO HARDENED PLANS (TRANSFORM + SERVE)

OUT · ONE SPEC · ONE END-TO-END EVAL

Take this fork when the pieces only make sense together — a tightly-coupled system where the trust boundary wraps the whole. Feed the two hardened swimlane plans (medallion + serving) into a spec framework — **SpecKit**, **OpenSpec**, or **AgentSpec** — which lifts the sketch into one coherent, coupled specification the engine builds against in one piece.

LIFT

Plans → spec

the two swimlane plans (transform + serve) become one SpecKit / OpenSpec / AgentSpec document.

COUPLE

One system

pieces only make sense together — the boundary wraps the whole, not each part.

BIND EVAL

End-to-end

one eval: sources → gold → the serving layer answers the question, whole-system.

GATE

One coherent spec exists, the whole system is its unit of trust, and a single **end-to-end eval** defines done — the gold tables build from the sources and the serving layer returns the answer, verified as one pipeline. This fork then enters the shared gate at Execution (Pass 7).

FORK B · TASK-DRIVEN · MANY ATOMIC UNITS

The other path — for loosely-coupled units that each stand alone (individual transformation models, individual serving endpoints) — takes **5B Tasking** below, then reconverges with 5A at **Harness (6)** → **Execute (7)**: cut the plans into atomic, eval-backed tasks, fit a harness to exactly those tasks, then build each task until its own eval is green. The trust boundary wraps *each unit*; verification is **per-unit runnable evals**.

PASS 5B · THE FORK STEP · ALTITUDE: ATOMIC UNIT

TASKING · TASK-DRIVEN · CUT THE WORK INTO EVAL-BACKED UNITS

5B · Tasking — atomic, self-contained, engine-agnostic.

ENGINE · TASK-SPEC SKILL	IN · THE TWO HARDENED PLANS	OUT · TASKS/* PER PLAN, EACH WITH AN EVAL

Cut the two hardened plans into indivisible units. Each task is one thing (“build the silver fact model”, “the orders serving endpoint”), names the work and paths — not the harness agent — and carries a runnable eval that defines done. A task-spec is self-contained and vendor-portable.

CUT

Atomize

one indivisible unit per task — “build the silver fact model”, “one serving endpoint”.

DESCRIBE

Tech, not agent

name the work + paths, not the harness agent — that comes next.

BIND EVAL

Done = ?

a runnable eval defines done — the model test passes, the endpoint returns N rows.

GATE

Every task carries a runnable eval — no eval, not a task yet. Task-spec is **self-contained** (its own architect + scripts) and **vendor-portable** (Claude, Codex, Kimi, or manual). The spec is engine-agnostic; the eval is engine-universal.

PASS 6 OF 7 · ALTITUDE: CONTROL LAYER

HARNESS · GENERATE THE BUILD-SYSTEM THE TASKS NEED

Build the rails the tasks will ride on.

ENGINE · AGENTS-KBS-TECH-STACK SKILL | IN · THE TASKS (THEY NAME WHAT THEY NEED)

OUT · .CLAUDE/ — AGENTS · KBS · RULES · CLAUDE.MD

Read what the tasks need — they name a warehouse, a transform tool, a serving layer — and scaffold exactly those: paired architect + developer agents per tech, grounded in real docs, plus KBs, rules, and a cross-tool contract. **Tasking before Harness is on purpose** — the tasks are the requirements; the harness is fitted to exactly what they need, not built speculatively.

READ NEEDS

Scope

the tasks name their tech — warehouse, transform, serving — scaffold exactly those.

SCAFFOLD

Agents + KBs

paired architect + developer per tech, grounded in real docs.

EMIT

Cross-tool

AGENTS.md · Cursor · Copilot — every engine inherits one contract.

GATE

The control layer stands; agents are grounded. **Tasking before Harness on purpose** — the tasks are the requirements; the harness is fitted to exactly what they need, not built speculatively. The harness is the one artifact that outlives the project. (See Appendix A.)

PASS 7 OF 7 · ALTITUDE: CODE

EXECUTION · BUILD UNTIL THE EVALS GO GREEN

The loop closes when the eval passes.

ENGINE · WORKFLOWS · AGENTSPEC · KIMI CRANK

IN · TASKS + GROUNDED HARNESS

OUT · MERGED CODE (TRANSFORM + SERVING)

Dispatch each task to the right engine — the crank engine builds the mechanical models, a reviewer judges, an adversary checks. Run each task's eval (the model test, the gold query, the endpoint responds). Loop until green, then merge. Automate this gate and the human steps out — that is the dark factory.

DISPATCH

Right engine

the crank engine builds the mechanical models · a reviewer judges · an adversary checks.

RUN EVAL

Green?

each task's eval runs — the model test, the gold query, the endpoint responds.

LOOP / MERGE

Until green

re-run until the eval passes — then merge.

GATE

eval passes = merged. Each engine in its role; each task looped until its eval is green. Automate this gate and the human steps out — that is the dark factory.

A · Why Harness Is Pass 6, Not Earlier

A natural instinct is to build the harness *before* choosing the fork (plan-driven vs task-driven). The spine puts Harness last but one — at Pass 6, after Tasking — and that ordering is correct. Here is the evidence, not the opinion.

What the fork actually decides

The fork sets where the **trust boundary** sits, and therefore the **unit of verification**: Fork A verifies with one end-to-end test; Fork B verifies with per-unit runnable evals. The harness's whole job is to make that verification runnable — so it cannot be built until the verification shape is known.

The two orderings, costed

	SPINE: FORK → TASK → HARNESS	HARNESS BEFORE FORK
Harness scope known?	Yes — tasks name what they touch (warehouse, transform, serving); scaffold exactly those	No — guess which agents / KBs to build before the units exist
Verification shape decided?	Yes — fork already chose end-to-end vs per-unit; harness inherits it	No — build rails without knowing if the eval is 1 test or N
Risk profile	Low — harness is the last thing built and outlives the project	High — speculative scaffold, the exact anti-pattern the spine names
If the fork flips	No rework — fork is upstream of both task and harness	Rebuild the harness when the boundary moves

The decisive point

Pass 5 (Tasking) is the literal *input* to Pass 6 — the Harness pass reads `IN · THE TASKS (THEY NAME WHAT THEY NEED)`. You cannot fit rails to tasks that don't exist yet, so Harness-before-Task isn't just riskier — it is missing its own input. And because the fork changes the unit of verification, building the harness first means committing to a verification shape before the fork has chosen it. That is backwards: the harness is downstream of the decision it exists to serve.

The one legitimate version of the instinct

> **GENERIC RAILS EARLY · FITTED RAILS AT PASS 6**
>
> Distinguish the **generic base harness** (repo conventions, lint, CI, the empty `.claude/` skeleton — i.e. the agents-kbs-tech-stack skill itself) from the **task-fitted harness** (the specific agents + KBs the tasks demand). A thin, reusable base can absolutely pre-exist any single project. What must not precede the fork is the *fitted* harness. Generic rails can be early; fitted rails stay at Pass 6.

VERDICT

Keep **Fork → Task → Harness**. Harness scope and verification shape are both outputs of the fork and tasking, so building the harness earlier is the speculative-scaffold anti-pattern the method explicitly warns against. The sequence is canonical.